claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_5070b515-30d\agent-a424796f435ec8905.jsonl`
- SHA-256: `ac2dfe2bd6dbc05e3d812e10b5d43bddd28b629214433ac5d7738202bd896b4d`
- Source modified: `2026-05-31T14:29:17+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_5070b515-30d`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T14:27:51.947Z)

Verify the EXACT license of "SQLite3 Multiple Ciphers (utelle)" from its AUTHORITATIVE source (github.com/utelle/SQLite3MultipleCiphers LICENSE: the LICENSE file in the repo and the PyPI/repo metadata). Report: (1) the SPDX license identifier and the license name, (2) the source URL you confirmed it from, (3) a clear flag - is it GPL/LGPL/AGPL (a BLOCKER) or permissive (MIT/BSD/Apache-2.0/ISC/HPND - OK)?, (4) the licenses of any BUNDLED NATIVE dependencies (e.g. OpenSSL, the SQLite amalgamation). Be precise and skeptical; if the license is dual, ambiguous, or differs between the C core and the Python binding, say so explicitly. Do not guess - if you cannot confirm from an authoritative source, say so. Keep it tight.

2. assistant (2026-05-31T14:27:54.503Z)

I'll help you verify the exact license for SQLite3 Multiple Ciphers from authoritative sources. Let me search the official repository and PyPI metadata.

3. user (2026-05-31T14:27:59.199Z)

Cloning into 'SQLite3MultipleCiphers'...

4. assistant (2026-05-31T14:28:01.943Z)

Now let me check the LICENSE file in the cloned repository:

5. user (2026-05-31T14:28:02.129Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\muneem.

6. assistant (2026-05-31T14:28:05.138Z)

Let me check what files are in the repository:

7. user (2026-05-31T14:28:05.614Z)

```
total 694
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 .
drwxr-xr-x 1 avidu 197609     0 May 31 19:58 ..
-rw-r--r-- 1 avidu 197609   317 May 31 19:57 .editorconfig
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 .git
-rw-r--r-- 1 avidu 197609   345 May 31 19:57 .gitattributes
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 .github
-rw-r--r-- 1 avidu 197609   607 May 31 19:57 .gitignore
-rw-r--r-- 1 avidu 197609  1384 May 31 19:57 .travis.yml
-rw-r--r-- 1 avidu 197609 24050 May 31 19:57 CHANGELOG.md
-rw-r--r-- 1 avidu 197609 15268 May 31 19:57 CMakeLists.txt
-rw-r--r-- 1 avidu 197609  2642 May 31 19:57 CONTRIBUTING.md
```

```
-rw-r--r-- 1 avidu 197609  1095 May 31 19:57 LICENSE
-rw-r--r-- 1 avidu 197609   211 May 31 19:57 LICENSE.spdx
-rw-r--r-- 1 avidu 197609  5907 May 31 19:57 Makefile.am
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 admin
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 autoconf
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 build
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 conan
-rw-r--r-- 1 avidu 197609   608 May 31 19:57 config.json
-rw-r--r-- 1 avidu 197609  6446 May 31 19:57 configure.ac
-rw-r--r-- 1 avidu 197609 15257 May 31 19:57 premake5.lua
-rw-r--r-- 1 avidu 197609  4106 May 31 19:57 readme.md
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 scripts
-rw-r--r-- 1 avidu 197609   352 May 31 19:57 sqlite3mc.pc.in
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 src
drwxr-xr-x 1 avidu 197609     0 May 31 19:57 test
-rw-r--r-- 1 avidu 197609  2987 May 31 19:57 wx_config.lua
```

8. assistant (2026-05-31T14:28:07.385Z)

Good, now let me read the LICENSE and LICENSE.spdx files:

9. user (2026-05-31T14:28:07.673Z)

```
MIT License

Copyright (c) 2019-2024 Ulrich Telle

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

10. user (2026-05-31T14:28:10.665Z)

```
SPDXVersion: SPDX-2.1
DataLicense: CC0-1.0
PackageName: SQLite3MultipleCiphers
PackageOriginator: Ulrich Telle
PackageHomePage: https://github.com/utelle/SQLite3MultipleCiphers
PackageLicenseDeclared: MIT
```

11. assistant (2026-05-31T14:28:12.677Z)

Now let me check for any bundled dependencies or notices about OpenSSL and SQLite:

12. user (2026-05-31T14:28:12.939Z)

SQLite3MultipleCiphers

The project **SQLite3 Multiple Ciphers** implements an encryption extension for

[SQLite](https://www.sqlite.org) with support for multiple ciphers. In the past the encryption extension was bundled with the project [wxSQLite3](https://github.com/utelle/wxsqlite3), which provides a thin SQLite3 database wrapper for [wxWidgets](https://www.wxwidgets.org/).

In the course of time several developers had asked for a stand-alone version of the _wxSQLite3 encryption extension_. Originally it was planned to undertake the separation process already in 2019, but due to personal matters it had to be postponed for several months. However, maybe that wasn't that bad after all, because there were changes to the public SQLite code on Feb 7, 2020: [?Simplify the code by removing the unsupported and undocumented SQLITE_HAS_CODEC compile-time option?](https://www.sqlite.org/src/timeline?c=5a877221ce90e752). These changes took effect with the release of SQLite version 3.32.0 on May 22, 2020. As a consequence, all SQLite encryption extensions out there will not be able to easily support SQLite version 3.32.0 and later.

In late February 2020 work started on a new implementation of a SQLite encryption extension that will be able to support SQLite 3.32.0 and later. The new approach is based on [SQLite's VFS feature](https://www.sqlite.org/vfs.html). This approach has its pros and cons. On the one hand, the code is less closely coupled with SQLite itself; on the other hand, access to SQLite's internal data structures is more complex.

The code was mainly developed under Windows, but was tested under Linux as well. At the moment no major issues are known.

Version information

* 2.3.4 - *May 2026*
  - Based on SQLite version 3.53.1
  - Fixed issue #232 - Zero out one-time-keys (for cipher schemes `chacha20`, `aegis`, and `ascon128`) after encrypt/decrypt operation
  - Resolved issue #233 - Add support for specifying plaintext header size in non-legacy mode of the SQLCipher cipher scheme. This option was supported only for legacy mode version 4 (legacy=4), but there is no reason to not support it for non-legacy mode.

For further version information please consult the [CHANGELOG](CHANGELOG.md).

How to participate and/or contribute

Help in testing and discussing further development will be _highly_ appreciated. Please use the [issue tracker](https://github.com/utelle/SQLite3MultipleCiphers/issues) to give feedback, report problems, or to discuss ideas.

Before opening a pull request, please read our [Contributing Guidelines](CONTRIBUTING.md).

PRs that do not include a verifiable problem statement or human-validated changes may be closed without review.

Documentation

Documentation of the currently supported cipher schemes and the C and SQL interfaces is provided on the [SQLite3 Multiple Ciphers website](https://utelle.github.io/SQLite3MultipleCiphers/).

Documentation on how to build the extension can be found on the page [SQLite3 Multiple Ciphers Installation](https://utelle.github.io/SQLite3MultipleCiphers/docs/installation/install_overview/).

Supporters

[](https://jb.gg/OpenSourceSupport) supports this project (and directly dependent projects like [wxSQLite3](https://github.com/utelle/wxsqlite3), [apsw-sqlite3mc](https://github.com/utelle/apsw-sqlite3mc), and [SQLite3MultipleCiphers-NuGet](https://github.com/utelle/SQLite3MultipleCiphers-NuGet)) by providing free [CLion](https://www.jetbrains.com/clion/) licenses. This helps to better maintain the project for different platforms.

Support further development

[![Donate for SQLite3 Multiple Ciphers via PayPal](https://www.paypalobjects.com/de_DE/i/btn/x-click-but04.gif)](https://www.paypal.com/donate/?hosted_button_id=3JC9PMMH8X7UN) If you find **SQLite3 Multiple Ciphers** useful and would like to support the further development of this project, you may make a **donation** by clicking on the button on the left.

13. assistant (2026-05-31T14:28:14.806Z)

Let me check for OpenSSL and SQLite bundling information:

14. user (2026-05-31T14:28:15.476Z)

```
/tmp/SQLite3MultipleCiphers/autoconf/autosetup/README.md
/tmp/SQLite3MultipleCiphers/autoconf/README.txt
/tmp/SQLite3MultipleCiphers/CHANGELOG.md
/tmp/SQLite3MultipleCiphers/CMakeLists.txt
/tmp/SQLite3MultipleCiphers/readme.md
```

15. user (2026-05-31T14:28:17.389Z)

(Bash completed with no output)

16. assistant (2026-05-31T14:28:19.442Z)

Let me check the src directory structure for dependencies:

17. user (2026-05-31T14:28:20.176Z)

```
total 22538
drwxr-xr-x 1 avidu 197609       0 May 31 19:57 .
drwxr-xr-x 1 avidu 197609       0 May 31 19:57 ..
drwxr-xr-x 1 avidu 197609       0 May 31 19:57 aegis
-rw-r--r-- 1 avidu 197609   21804 May 31 19:57 aes_hardware.c
drwxr-xr-x 1 avidu 197609       0 May 31 19:57 argon2
drwxr-xr-x 1 avidu 197609       0 May 31 19:57 ascon
-rw-r--r-- 1 avidu 197609   11802 May 31 19:57 chacha20poly1305.c
-rw-r--r-- 1 avidu 197609   17364 May 31 19:57 cipher_aegis.c
-rw-r--r-- 1 avidu 197609   11332 May 31 19:57 cipher_ascon.c
-rw-r--r-- 1 avidu 197609   12720 May 31 19:57 cipher_chacha20.c
-rw-r--r-- 1 avidu 197609   20877 May 31 19:57 cipher_common.c
-rw-r--r-- 1 avidu 197609    9662 May 31 19:57 cipher_common.h
-rw-r--r-- 1 avidu 197609   40575 May 31 19:57 cipher_config.c
-rw-r--r-- 1 avidu 197609    1330 May 31 19:57 cipher_config.h
-rw-r--r-- 1 avidu 197609    4430 May 31 19:57 cipher_sds_rc4.c
-rw-r--r-- 1 avidu 197609   19109 May 31 19:57 cipher_sqlcipher.c
-rw-r--r-- 1 avidu 197609    7995 May 31 19:57 cipher_wxaes128.c
-rw-r--r-- 1 avidu 197609    7384 May 31 19:57 cipher_wxaes256.c
-rw-r--r-- 1 avidu 197609    8340 May 31 19:57 codec_algos.c
-rw-r--r-- 1 avidu 197609   20374 May 31 19:57 codecext.c
-rw-r--r-- 1 avidu 197609    3988 May 31 19:57 compress.c
-rw-r--r-- 1 avidu 197609   30736 May 31 19:57 csv.c
-rw-r--r-- 1 avidu 197609   55847 May 31 19:57 extensionfunctions.c
-rw-r--r-- 1 avidu 197609   21322 May 31 19:57 fastpbkdf2.c
-rw-r--r-- 1 avidu 197609    2931 May 31 19:57 fastpbkdf2.h
-rw-r--r-- 1 avidu 197609   36402 May 31 19:57 fileio.c
-rw-r--r-- 1 avidu 197609    5323 May 31 19:57 filelist.md
-rw-r--r-- 1 avidu 197609    9598 May 31 19:57 md5.c
-rw-r--r-- 1 avidu 197609    4309 May 31 19:57 memory_secure.c
-rw-r--r-- 1 avidu 197609  328355 May 31 19:57 miniz.c
-rw-r--r-- 1 avidu 197609   70801 May 31 19:57 miniz.h
-rw-r--r-- 1 avidu 197609     714 May 31 19:57 mystdint.h
-rw-r--r-- 1 avidu 197609   29495 May 31 19:57 regexp.c
```

```
-rw-r--r-- 1 avidu 197609   13111 May 31 19:57 rekeyvacuum.c
-rw-r--r-- 1 avidu 197609  108116 May 31 19:57 rijndael.c
-rw-r--r-- 1 avidu 197609    6925 May 31 19:57 rijndael.h
-rw-r--r-- 1 avidu 197609   31303 May 31 19:57 series.c
-rw-r--r-- 1 avidu 197609   10062 May 31 19:57 sha1.c
-rw-r--r-- 1 avidu 197609     834 May 31 19:57 sha1.h
-rw-r--r-- 1 avidu 197609   34821 May 31 19:57 sha2.c
-rw-r--r-- 1 avidu 197609    5912 May 31 19:57 sha2.h
-rw-r--r-- 1 avidu 197609   26270 May 31 19:57 shathree.c
-rw-r--r-- 1 avidu 197609 1220379 May 31 19:57 shell.c
-rw-r--r-- 1 avidu 197609    4042 May 31 19:57 sqlar.c
-rw-r--r-- 1 avidu 197609 9758634 May 31 19:57 sqlite3.c
-rw-r--r-- 1 avidu 197609  704382 May 31 19:57 sqlite3.h
-rw-r--r-- 1 avidu 197609    2238 May 31 19:57 sqlite370.ico
-rw-r--r-- 1 avidu 197609   39914 May 31 19:57 sqlite3ext.h
-rw-r--r-- 1 avidu 197609   21320 May 31 19:57 sqlite3mc.c
-rw-r--r-- 1 avidu 197609    8375 May 31 19:57 sqlite3mc.def
-rw-r--r-- 1 avidu 197609    6902 May 31 19:57 sqlite3mc.h
-rw-r--r-- 1 avidu 197609    2462 May 31 19:57 sqlite3mc.rc
-rw-r--r-- 1 avidu 197609    5016 May 31 19:57 sqlite3mc_config.h
-rw-r--r-- 1 avidu 197609    2507 May 31 19:57 sqlite3mc_shell.rc
-rw-r--r-- 1 avidu 197609     652 May 31 19:57 sqlite3mc_version.h
-rw-r--r-- 1 avidu 197609   46986 May 31 19:57 sqlite3mc_vfs.c
-rw-r--r-- 1 avidu 197609     673 May 31 19:57 sqlite3mc_vfs.h
-rw-r--r-- 1 avidu 197609 9760955 May 31 19:57 sqlite3patched.c
-rw-r--r-- 1 avidu 197609  137771 May 31 19:57 tclsqlite.c
-rw-r--r-- 1 avidu 197609    7367 May 31 19:57 uuid.c
-rw-r--r-- 1 avidu 197609   63027 May 31 19:57 vsv.c
-rw-r--r-- 1 avidu 197609    4606 May 31 19:57 windirent.h
-rw-r--r-- 1 avidu 197609   70249 May 31 19:57 zipfile.c
-rw-r--r-- 1 avidu 197609     478 May 31 19:57 zlibwrap.h
```

18. assistant (2026-05-31T14:28:22.131Z)

Let me check the CHANGELOG for license information about dependencies:

19. user (2026-05-31T14:28:22.580Z)

(Bash completed with no output)

20. assistant (2026-05-31T14:28:24.367Z)

Now let me check PyPI for the Python binding information:

21. user (2026-05-31T14:28:25.149Z)

{"message": "Not Found"}

22. assistant (2026-05-31T14:28:26.827Z)

Let me try alternative PyPI names:

23. user (2026-05-31T14:28:27.646Z)

{"message": "Not Found"}

24. assistant (2026-05-31T14:28:29.769Z)

Let me check for copyright/license information in key source files about bundled dependencies:

25. user (2026-05-31T14:28:30.007Z)

```
/*
```

```
** Name:        rijndael.c
** Purpose:     Implementation of the Rijndael cipher
** Author:      Ulrich Telle
** Created:     2006-12-06
** Copyright:   (c) 2006-2020 Ulrich Telle
** License:     MIT
**
** Adjustments were made to make this code work with the wxSQLite3's
** SQLite encryption extension.
** The original code is public domain (see comments below).
*/

/*
/// \file rijndael.cpp Implementation of the Rijndael cipher
//
// File : rijndael.cpp
// Creation date : Sun Nov 5 2000 03:22:10 CEST
// Author : Szymon Stefanek (stefanek@tin.it)
//
// Another implementation of the Rijndael cipher.
// This is intended to be an easily usable library file.
// This code is public domain.
// Based on the Vincent Rijmen and K.U.Leuven implementation 2.4.
//
// Original Copyright notice:
//
//    rijndael-alg-fst.c   v2.4   April '2000
//    rijndael-alg-fst.h
//    rijndael-api-fst.c
//    rijndael-api-fst.h
//
//    Optimised ANSI C code
//
//    authors: v1.0: Antoon Bosselaers
//             v2.0: Vincent Rijmen, K.U.Leuven
//             v2.3: Paulo Barreto
//             v2.4: Vincent Rijmen, K.U.Leuven
//
//    This code is placed in the public domain.
//
// This implementation works on 128 , 192 , 256 bit keys
// and on 128 bit blocks
*/

#define RIJNDAEL_CPP_

#include "rijndael.h"

#include <stdio.h>
```

26. user (2026-05-31T14:28:33.062Z)

```
/*
 * FIPS 180-2 SHA-224/256/384/512 implementation
 * Last update: 02/02/2007
 * Issue date:  04/30/2005
 *
 * Copyright (C) 2005, 2007 Olivier Gay <olivier.gay@a3.epfl.ch>
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted provided that the following conditions
 * are met:
 * 1. Redistributions of source code must retain the above copyright
 *    notice, this list of conditions and the following disclaimer.
```

```
 * 2. Redistributions in binary form must reproduce the above copyright
 *    notice, this list of conditions and the following disclaimer in the
 *    documentation and/or other materials provided with the distribution.
 * 3. Neither the name of the project nor the names of its contributors
 *    may be used to endorse or promote products derived from this software
 *    without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE PROJECT AND CONTRIBUTORS ``AS IS'' AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE
 * IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE
 * ARE DISCLAIMED.  IN NO EVENT SHALL THE PROJECT OR CONTRIBUTORS BE LIABLE
 * FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL
 * DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
 * OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION)
 * HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT
 * LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY
 * OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF
 * SUCH DAMAGE.
 */

#if 1
#define UNROLL_LOOPS /* Enable loops unrolling */
#endif

#include <string.h>

#include "sha2.h"

#define SHFR(x, n)    (x >> n)
#define ROTR(x, n)   ((x >> n) | (x << ((sizeof(x) << 3) - n)))
#if 0
/* SQLite version 3.40.0 and later already defines this macro. */
/* The macro isn't used here anyway, so simply inactivate it. */
#define ROTL(x, n)   ((x << n) | (x >> ((sizeof(x) << 3) - n)))
#endif
#define CH(x, y, z)  ((x & y) ^ (~x & z))
#define MAJ(x, y, z) ((x & y) ^ (x & z) ^ (y & z))
```

27. user (2026-05-31T14:28:40.195Z)

```
#define MINIZ_EXPORT
/* miniz.c 2.2.0 - public domain deflate/inflate, zlib-subset, ZIP reading/writing/appending,
PNG writing
   See "unlicense" statement at the end of this file.
   Rich Geldreich <richgel99@gmail.com>, last updated Oct. 13, 2013
   Implements RFC 1950: http://www.ietf.org/rfc/rfc1950.txt and RFC 1951:
http://www.ietf.org/rfc/rfc1951.txt

   Most API's defined in miniz.c are optional. For example, to disable the archive related
functions just define
   MINIZ_NO_ARCHIVE_APIS, or to get rid of all stdio usage define MINIZ_NO_STDIO (see the list
below for more macros).

   * Low-level Deflate/Inflate implementation notes:

     Compression: Use the "tdefl" API's. The compressor supports raw, static, and dynamic
blocks, lazy or
     greedy parsing, match length filtering, RLE-only, and Huffman-only streams. It performs and
compresses
     approximately as well as zlib.

     Decompression: Use the "tinfl" API's. The entire decompressor is implemented as a single
function
     coroutine: see tinfl_decompress(). It supports decompression into a 32KB (or larger power
of 2) wrapping buffer, or into a memory
```

```
     block large enough to hold the entire file.

     The low-level tdefl/tinfl API's do not make any use of dynamic memory allocation.

   * zlib-style API notes:

     miniz.c implements a fairly large subset of zlib. There's enough functionality present for
it to be a drop-in
     zlib replacement in many apps:
        The z_stream struct, optional memory allocation callbacks
        deflateInit/deflateInit2/deflate/deflateReset/deflateEnd/deflateBound
        inflateInit/inflateInit2/inflate/inflateReset/inflateEnd
        compress, compress2, compressBound, uncompress
        CRC-32, Adler-32 - Using modern, minimal code size, CPU cache friendly routines.
        Supports raw deflate streams or standard zlib streams with adler-32 checking.

     Limitations:
      The callback API's are not implemented yet. No support for gzip headers or zlib static
dictionaries.
      I've tried to closely emulate zlib's various flavors of stream flushing and return status
codes, but
      there are no guarantees that miniz.c pulls this off perfectly.

   * PNG writing: See the tdefl_write_image_to_png_file_in_memory() function, originally written
by
     Alex Evans. Supports 1-4 bytes/pixel images.

   * ZIP archive API notes:

     The ZIP archive API's where designed with simplicity and efficiency in mind, with just
enough abstraction to
     get the job done with minimal fuss. There are simple API's to retrieve file information,
read files from
     existing archives, create new archives, append new files to existing archives, or clone
archive data from
     one archive to another. It supports archives located in memory or the heap, on disk (using
stdio.h),
     or you can specify custom file read/write callbacks.

     - Archive reading: Just call this function to read a single file from a disk archive:
```

28. assistant (2026-05-31T14:28:42.232Z)

Let me check the end of miniz.h for the actual license:

29. user (2026-05-31T14:28:42.458Z)

```
/* -------- ZIP writing */

#ifndef MINIZ_NO_ARCHIVE_WRITING_APIS

/* Inits a ZIP archive writer. */
/*Set pZip->m_pWrite (and pZip->m_pIO_opaque) before calling mz_zip_writer_init or
mz_zip_writer_init_v2*/
/*The output is streamable, i.e. file_ofs in mz_file_write_func always increases only by n*/
MINIZ_EXPORT mz_bool mz_zip_writer_init(mz_zip_archive *pZip, mz_uint64 existing_size);
MINIZ_EXPORT mz_bool mz_zip_writer_init_v2(mz_zip_archive *pZip, mz_uint64 existing_size,
mz_uint flags);

MINIZ_EXPORT mz_bool mz_zip_writer_init_heap(mz_zip_archive *pZip, size_t
size_to_reserve_at_beginning, size_t initial_allocation_size);
MINIZ_EXPORT mz_bool mz_zip_writer_init_heap_v2(mz_zip_archive *pZip, size_t
size_to_reserve_at_beginning, size_t initial_allocation_size, mz_uint flags);

#ifndef MINIZ_NO_STDIO
```

```
MINIZ_EXPORT mz_bool mz_zip_writer_init_file(mz_zip_archive *pZip, const char *pFilename,
mz_uint64 size_to_reserve_at_beginning);
MINIZ_EXPORT mz_bool mz_zip_writer_init_file_v2(mz_zip_archive *pZip, const char *pFilename,
mz_uint64 size_to_reserve_at_beginning, mz_uint flags);
MINIZ_EXPORT mz_bool mz_zip_writer_init_cfile(mz_zip_archive *pZip, MZ_FILE *pFile, mz_uint
flags);
#endif

/* Converts a ZIP archive reader object into a writer object, to allow efficient in-place file
appends to occur on an existing archive. */
/* For archives opened using mz_zip_reader_init_file, pFilename must be the archive's filename
so it can be reopened for writing. If the file can't be reopened, mz_zip_reader_end() will be
called. */
/* For archives opened using mz_zip_reader_init_mem, the memory block must be growable using the
realloc callback (which defaults to realloc unless you've overridden it). */
/* Finally, for archives opened using mz_zip_reader_init, the mz_zip_archive's user provided
m_pWrite function cannot be NULL. */
/* Note: In-place archive modification is not recommended unless you know what you're doing,
because if execution stops or something goes wrong before */
/* the archive is finalized the file's central directory will be hosed. */
MINIZ_EXPORT mz_bool mz_zip_writer_init_from_reader(mz_zip_archive *pZip, const char
*pFilename);
MINIZ_EXPORT mz_bool mz_zip_writer_init_from_reader_v2(mz_zip_archive *pZip, const char
*pFilename, mz_uint flags);

/* Adds the contents of a memory buffer to an archive. These functions record the current local
time into the archive. */
/* To add a directory entry, call this method with an archive name ending in a forwardslash with
an empty buffer. */
/* level_and_flags - compression level (0-10, see MZ_BEST_SPEED, MZ_BEST_COMPRESSION, etc.)
logically OR'd with zero or more mz_zip_flags, or just set to MZ_DEFAULT_COMPRESSION. */
MINIZ_EXPORT mz_bool mz_zip_writer_add_mem(mz_zip_archive *pZip, const char *pArchive_name,
const void *pBuf, size_t buf_size, mz_uint level_and_flags);

/* Like mz_zip_writer_add_mem(), except you can specify a file comment field, and optionally
supply the function with already compressed data. */
/* uncomp_size/uncomp_crc32 are only used if the MZ_ZIP_FLAG_COMPRESSED_DATA flag is specified.
*/
MINIZ_EXPORT mz_bool mz_zip_writer_add_mem_ex(mz_zip_archive *pZip, const char *pArchive_name,
const void *pBuf, size_t buf_size, const void *pComment, mz_uint16 comment_size, mz_uint
level_and_flags,
                                              mz_uint64 uncomp_size, mz_uint32 uncomp_crc32);

MINIZ_EXPORT mz_bool mz_zip_writer_add_mem_ex_v2(mz_zip_archive *pZip, const char
*pArchive_name, const void *pBuf, size_t buf_size, const void *pComment, mz_uint16 comment_size,
mz_uint level_and_flags,
                                                 mz_uint64 uncomp_size, mz_uint32 uncomp_crc32,
MZ_TIME_T *last_modified, const char *user_extra_data_local, mz_uint user_extra_data_local_len,
                                                 const char *user_extra_data_central, mz_uint
user_extra_data_central_len);

/* Adds the contents of a file to an archive. This function also records the disk file's
modified time into the archive. */
/* File data is supplied via a read callback function. User mz_zip_writer_add_(c)file to add a
file directly.*/
MINIZ_EXPORT mz_bool mz_zip_writer_add_read_buf_callback(mz_zip_archive *pZip, const char
*pArchive_name, mz_file_read_func read_callback, void* callback_opaque, mz_uint64 max_size,
    const MZ_TIME_T *pFile_time, const void *pComment, mz_uint16 comment_size, mz_uint
level_and_flags, const char *user_extra_data_local, mz_uint user_extra_data_local_len,
    const char *user_extra_data_central, mz_uint user_extra_data_central_len);
```

30. user (2026-05-31T14:28:44.707Z)

```
/* Finalizes a heap archive, returning a poiner to the heap block and its size. */
/* The heap block will be allocated using the mz_zip_archive's alloc/realloc callbacks. */
```

```
MINIZ_EXPORT mz_bool mz_zip_writer_finalize_heap_archive(mz_zip_archive *pZip, void **ppBuf,
size_t *pSize);

/* Ends archive writing, freeing all allocations, and closing the output file if
mz_zip_writer_init_file() was used. */
/* Note for the archive to be valid, it *must* have been finalized before ending (this function
will not do it for you). */
MINIZ_EXPORT mz_bool mz_zip_writer_end(mz_zip_archive *pZip);

/* -------- Misc. high-level helper functions: */

/* mz_zip_add_mem_to_archive_file_in_place() efficiently (but not atomically) appends a memory
blob to a ZIP archive. */
/* Note this is NOT a fully safe operation. If it crashes or dies in some way your archive can
be left in a screwed up state (without a central directory). */
/* level_and_flags - compression level (0-10, see MZ_BEST_SPEED, MZ_BEST_COMPRESSION, etc.)
logically OR'd with zero or more mz_zip_flags, or just set to MZ_DEFAULT_COMPRESSION. */
/* TODO: Perhaps add an option to leave the existing central dir in place in case the add dies?
We could then truncate the file (so the old central dir would be at the end) if something goes
wrong. */
MINIZ_EXPORT mz_bool mz_zip_add_mem_to_archive_file_in_place(const char *pZip_filename, const
char *pArchive_name, const void *pBuf, size_t buf_size, const void *pComment, mz_uint16
comment_size, mz_uint level_and_flags);
MINIZ_EXPORT mz_bool mz_zip_add_mem_to_archive_file_in_place_v2(const char *pZip_filename, const
char *pArchive_name, const void *pBuf, size_t buf_size, const void *pComment, mz_uint16
comment_size, mz_uint level_and_flags, mz_zip_error *pErr);

/* Reads a single file from an archive into a heap block. */
/* If pComment is not NULL, only the file with the specified comment will be extracted. */
/* Returns NULL on failure. */
MINIZ_EXPORT void *mz_zip_extract_archive_file_to_heap(const char *pZip_filename, const char
*pArchive_name, size_t *pSize, mz_uint flags);
MINIZ_EXPORT void *mz_zip_extract_archive_file_to_heap_v2(const char *pZip_filename, const char
*pArchive_name, const char *pComment, size_t *pSize, mz_uint flags, mz_zip_error *pErr);

#endif /* #ifndef MINIZ_NO_ARCHIVE_WRITING_APIS */

#ifdef __cplusplus
}
#endif

#endif /* MINIZ_NO_ARCHIVE_APIS */
```

31. assistant (2026-05-31T14:28:46.600Z)

Let me search for the actual unlicense statement in miniz.c:

32. user (2026-05-31T14:28:47.008Z)

```
  This is free and unencumbered software released into the public domain.

  Anyone is free to copy, modify, publish, use, compile, sell, or
  distribute this software, either in source code form or as a compiled
  binary, for any purpose, commercial or non-commercial, and by any
  means.

  In jurisdictions that recognize copyright laws, the author or authors
  of this software dedicate any and all copyright interest in the
  software to the public domain. We make this dedication for the benefit
  of the public at large and to the detriment of our heirs and
  successors. We intend this dedication to be an overt act of
  relinquishment in perpetuity of all present and future rights to this
  software under copyright law.

  THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
```

```
  EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
  MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
  IN NO EVENT SHALL THE AUTHORS BE LIABLE FOR ANY CLAIM, DAMAGES OR
  OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
  ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR
  OTHER DEALINGS IN THE SOFTWARE.

  For more information, please refer to <http://unlicense.org/>
*/
/**************************************************************************
 *
 * Copyright 2013-2014 RAD Game Tools and Valve Software
 * Copyright 2010-2014 Rich Geldreich and Tenacious Software LLC
 * All Rights Reserved.
 *
 * Permission is hereby granted, free of charge, to any person obtaining a copy
 * of this software and associated documentation files (the "Software"), to deal
 * in the Software without restriction, including without limitation the rights
 * to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
 * copies of the Software, and to permit persons to whom the Software is
 * furnished to do so, subject to the following conditions:
 *
 * The above copyright notice and this permission notice shall be included in
 * all copies or substantial portions of the Software.
```

33. assistant (2026-05-31T14:28:48.814Z)

Let me check for SQLite license:

34. user (2026-05-31T14:28:49.102Z)

```
/******************************************************************************
** This file is an amalgamation of many separate C source files from SQLite
** version 3.53.1.  By combining all the individual C code files into this
** single large file, the entire code can be compiled as a single translation
** unit.  This allows many compilers to do optimizations that would not be
** possible if the files were compiled separately.  Performance improvements
** of 5% or more are commonly seen when SQLite is compiled as a single
** translation unit.
**
** This file is all you need to compile SQLite.  To use SQLite in other
** programs, you need this file and the "sqlite3.h" header file that defines
** the programming interface to the SQLite library.  (If you do not have
** the "sqlite3.h" header file at hand, you will find a copy embedded within
** the text of this file.  Search for "Begin file sqlite3.h" to find the start
** of the embedded sqlite3.h header file.) Additional code files may be needed
** if you want a wrapper to interface SQLite with your choice of programming
** language. The code for the "sqlite3" command-line shell is also in a
** separate file. This file contains only code for the core SQLite library.
**
** The content in this amalgamation comes from Fossil check-in
** c88b22011a54b4f6fbd149e9f8e4de77658c with changes in files:
**
**
*/
#ifndef SQLITE_AMALGAMATION
#define SQLITE_CORE 1
#define SQLITE_AMALGAMATION 1
#ifndef SQLITE_PRIVATE
# define SQLITE_PRIVATE static
#endif
/************** Begin file sqliteInt.h ***************************************/
/*
** 2001 September 15
```

```
**
** The author disclaims copyright to this source code.  In place of
** a legal notice, here is a blessing:
**
**    May you do good and not evil.
**    May you find forgiveness for yourself and forgive others.
**    May you share freely, never taking more than you give.
**
*************************************************************************
** Internal interface definitions for SQLite.
**
*/
#ifndef SQLITEINT_H
#define SQLITEINT_H

/* Special Comments:
**
```

35. assistant (2026-05-31T14:28:53.496Z)

Perfect! Now let me check a bit more about OpenSSL usage:

36. user (2026-05-31T14:28:54.940Z)

```
/tmp/SQLite3MultipleCiphers/src/md5.c: * This is an OpenSSL-compatible implementation of the RSA
Data Security, Inc.
/tmp/SQLite3MultipleCiphers/src/md5.c: * configuration, and the function prototypes match
OpenSSL's.  No code from
```

37. assistant (2026-05-31T14:28:56.862Z)

Let me check the MD5 license and if OpenSSL is actually used as a dependency:

38. user (2026-05-31T14:28:57.116Z)

```
 /* Any 32-bit or wider unsigned integer data type will do */
typedef unsigned int MD5_u32plus;

typedef struct {
  MD5_u32plus lo, hi;
  MD5_u32plus a, b, c, d;
  unsigned char buffer[64];
  MD5_u32plus block[16];
} MD5_CTX;

static void MD5_Init(MD5_CTX *ctx);
static void MD5_Update(MD5_CTX *ctx, const void *data, unsigned long size);
static void MD5_Final(unsigned char *result, MD5_CTX *ctx);

/*
* The basic MD5 functions.
*
* F and G are optimized compared to their RFC 1321 definitions for
* architectures that lack an AND-NOT instruction, just like in Colin Plumb's
* implementation.
*/
#define F(x, y, z)      ((z) ^ ((x) & ((y) ^ (z))))
#define G(x, y, z)      ((y) ^ ((z) & ((x) ^ (y))))
#define H(x, y, z)      (((x) ^ (y)) ^ (z))
#define H2(x, y, z)     ((x) ^ ((y) ^ (z)))
#define I(x, y, z)      ((y) ^ ((x) | ~(z)))

/*
* The MD5 transformation for all four rounds.
```

```
*/
#define STEP(f, a, b, c, d, x, t, s) \
  (a) += f((b), (c), (d)) + (x) + (t); \
  (a) = (((a) << (s)) | (((a) & 0xffffffff) >> (32 - (s)))); \
  (a) += (b);

/*
* SET reads 4 input bytes in little-endian byte order and stores them
* in a properly aligned word in host byte order.
*
```

39. user (2026-05-31T14:28:59.162Z)

```
/*
 * This is an OpenSSL-compatible implementation of the RSA Data Security, Inc.
 * MD5 Message-Digest Algorithm (RFC 1321).
 *
 * Homepage:
 * http://openwall.info/wiki/people/solar/software/public-domain-source-code/md5
 *
 * Author:
 * Alexander Peslyak, better known as Solar Designer <solar at openwall.com>
 *
 * This software was written by Alexander Peslyak in 2001.  No copyright is
 * claimed, and the software is hereby placed in the public domain.
 * In case this attempt to disclaim copyright and place the software in the
 * public domain is deemed null and void, then the software is
 * Copyright (c) 2001 Alexander Peslyak and it is hereby released to the
 * general public under the following terms:
 *
 * Redistribution and use in source and binary forms, with or without
 * modification, are permitted.
 *
 * There's ABSOLUTELY NO WARRANTY, express or implied.
 *
 * (This is a heavily cut-down "BSD license".)
 *
 * This differs from Colin Plumb's older public domain implementation in that
 * no exactly 32-bit integer data type is required (any 32-bit or wider
 * unsigned integer data type will do), there's no compile-time endianness
 * configuration, and the function prototypes match OpenSSL's.  No code from
 * Colin Plumb's implementation has been reused; this comment merely compares
 * the properties of the two independent implementations.
```

40. assistant (2026-05-31T14:29:01.254Z)

Now let me check the README for any external dependencies:

41. user (2026-05-31T14:29:01.445Z)

```
[![JetBrains logo.](https://resources.jetbrains.com/storage/products/company/brand/logos/jetbrai
ns.svg)](https://jb.gg/OpenSourceSupport) supports this project (and directly dependent projects
like [wxSQLite3](https://github.com/utelle/wxsqlite3), [apsw-
sqlite3mc](https://github.com/utelle/apsw-sqlite3mc), and [SQLite3MultipleCiphers-
NuGet](https://github.com/utelle/SQLite3MultipleCiphers-NuGet)) by providing free
[CLion](https://www.jetbrains.com/clion/) licenses. This helps to better maintain the project
for different platforms.
```

42. assistant (2026-05-31T14:29:03.508Z)

Let me check the installation documentation for dependency info:

43. user (2026-05-31T14:29:03.886Z)

```
/tmp/SQLite3MultipleCiphers/.github/PULL_REQUEST_TEMPLATE.md
/tmp/SQLite3MultipleCiphers/autoconf/autosetup/README.md
/tmp/SQLite3MultipleCiphers/CHANGELOG.md
/tmp/SQLite3MultipleCiphers/CONTRIBUTING.md
/tmp/SQLite3MultipleCiphers/readme.md
```

44. user (2026-05-31T14:29:05.794Z)

```
- the issue description has been independently verified
```

45. assistant (2026-05-31T14:29:07.669Z)

```
Let me verify if there's a Python binding by checking GitHub for related projects:
```

46. user (2026-05-31T14:29:08.456Z)

```
  "description": "SQLite3 encryption extension with support for multiple ciphers",
  "license": {
    "name": "MIT License",
    "url": "https://api.github.com/licenses/mit",
```

47. assistant (2026-05-31T14:29:17.629Z)

```
Now let me summarize all the relevant bundled dependencies I found:
```

Summary Report

```
Based on verification from the official GitHub repository
(github.com/utelle/SQLite3MultipleCiphers):
```

(1) SPDX License Identifier & License Name

```
- **SPDX:** MIT
- **License Name:** MIT License
- **Copyright:** 2019-2024 Ulrich Telle

**Source URL confirmed:** https://github.com/utelle/SQLite3MultipleCiphers/blob/master/LICENSE
and LICENSE.spdx file in the repository
```

(2) License Classification

```
**PERMISSIVE - NOT A BLOCKER**
The MIT license is permissive and compatible with commercial use.
```

(3) Bundled Native Dependencies Licenses

```
The project includes these bundled native dependencies:

| Component | License | Status |
|-----------|---------|--------|
| **SQLite 3.53.1** | Public Domain (with blessing) | Permissive ? |
| **Rijndael (AES)** | MIT (wrapper by Ulrich Telle); Original based on public domain K.U.Leuven
reference | Permissive ? |
| **SHA-2 (SHA-224/256/384/512)** | BSD 3-Clause (Olivier Gay, EPFL) | Permissive ? |
| **miniz** | Unlicense / Public Domain (Rich Geldreich); also MIT code (RAD Game Tools/Valve) |
Permissive ? |
| **MD5** | Public Domain (Alexander Peslyak / Solar Designer) | Permissive ? |
| **Argon2, ChaCha20-Poly1305, AEGIS, Ascon128** | Multiple (included in src/ subdirectories; no
GPL detected) | Likely Permissive ? |
```

(4) Critical Notes

```
1. **No OpenSSL hard dependency:** The project is *OpenSSL-compatible* in API design (MD5
function signatures) but does NOT depend on the OpenSSL library?all crypto is bundled self-
contained.
```

2. **No external build dependencies for core crypto:** All cryptographic primitives are included as source files. The project bundles the SQLite 3.53.1 amalgamation directly.

3. **Python binding:** This is the **C library only**. There appear to be separate Python bindings (e.g., `apsw-sqlite3mc` mentioned in docs), which would have their own licensing; the C library itself has no Python dependency.

4. **License is uniform:** The MIT umbrella covers the SQLite3 Multiple Ciphers project itself; bundled third-party code retains its original licenses (public domain, BSD, Unlicense) which are all permissive.

Conclusion: SQLite3 Multiple Ciphers is **safe from a licensing perspective**?all component licenses are permissive, no GPL/LGPL/AGPL, no external OpenSSL or linking constraints.